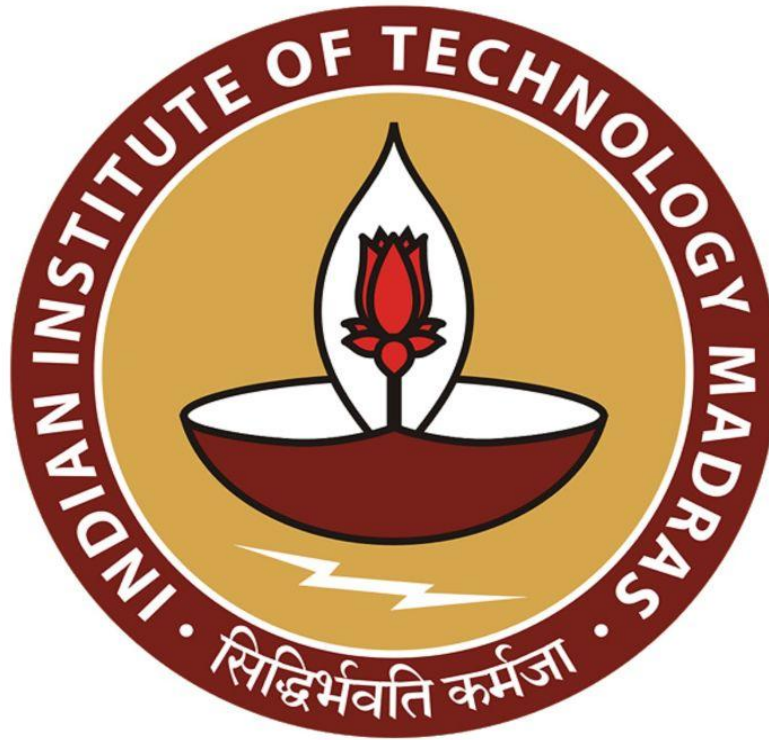




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology Madras
Gradient Descent

Hello, everyone, we've been looking at supervised learning specifically, we started looking at the problem of regression. And we posted it as a problem of linear regression where we wanted to search for linear functions that best fits the data in the sense of squared error. We looked at, we looked at the fact that linear regression can be solved in analytical form that is, you have a closed form solution in the sense that w^* can be written exactly as a function of x and y .

And then we looked at, we asked some questions about the nature of solution and what more can we understand about linear regression, towards that, we have already looked at the geometric view of what it means to say, w^* is of a particular form specifically $(xx^T)^+ xy$. What does it mean by that we have looked at the geometric interpretation. So, now, in this video, what we want to do is look at few more questions about linear regression and then try to address them.

(Refer Slide Time: 01:27)

COMPUTATIONAL CONSIDERATIONS

$w^* = (xx^T)^+ xy$

inverse computation
is expensive if d is large
 $O(d^3)$

IIT Madras
B.S. Degree

The first question that we will consider today is about the computational considerations with respect to linear regression. Computational considerations. So, one thing that is good about linear

regression is that, we have a closed form solution for this for the optimal w , which is of the form $(xx^T)^+ xy$. Now, it is a great thing that we can solve this as just an equation.

So, given data, I can just compute $(xx^T)^+ xy$, and then I get my w^* , which I later use to make my predictions on test data. However, the issue is that the issue that might crop up is how computationally expensive is it to compute this w^* ? Now, if you take a look at the form of this w^* , the main culprit here at least seems to be immediately this inverse or pseudo inverse. So, in general, you can assume this is an inverse also, that is fine. So, this inverse computation is expensive if d is large.

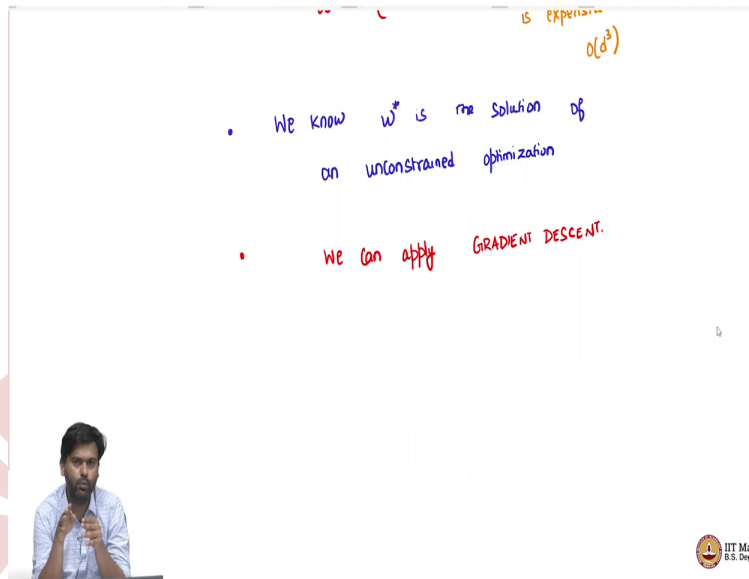
Remember, we made a similar comment when you were doing PCA that you wanted to compute the Eigen vectors of a covariance matrix and we said that the Eigen vector computation is typically hard, not hard so, it is computationally expensive of the order d^3 the inverse computation is exactly as hard as doing the Eigen vector computation, because if you know the Eigen vectors, you can actually compute the inverse very easily by just inverting the Eigen values and so on. So, the question is, what time does it take?

So, this also takes something like order of d^3 , maybe there are more specific, specialized special purpose algorithms, which can do it slightly faster than this but then in general, this is also going to take d^3 , where d is the number of features, so, remember, xx^T is of the form of $d \times d$ matrix.

So, so now, the question is if we cannot do this inverse, then how can we solve this problem. So, we know that this is the solution if we cannot compute it because d is large, maybe d is billion. So, you have billion features, so, you cannot invert that easily a billion $\times$ billion matrix, it is going to take too much computational effort.

So, the question is, is there a different way to achieve the same w^* without computing the inverse? So, this is a very practical question that one must ask because at the end of the day, it is not just the theoretical part that one needs to understand, of course, the theory, builds the foundation, but then what challenges might arise when you actually implement it in practice is also something to keep in mind.

(Refer Slide Time: 04:26)



Now, without computing the inverse, can we do something, well, we know that w^* is the solution of an unconstrained optimization problem. So, remember, w^* is just the minimizer of the sum of the squared error. So, the average of the squared errors of the prediction $w^T x_i - y_i$.

So, now because this is this does not, this is I mean, w^* can be any vector in d dimension, and it is the solution of an unconstrained optimization problem. Now, we can use general purpose tools to solve unconstrained optimization problems to solve for w^* also, and one of the most popular tools, general purpose tools to get w^* , or in general to get the minimizer of a function is the method of gradient descent.

Well, a very quick primer of what gradient descent is, in case you are not aware of it, gradient descent is an iterative way to find minimization minimizers of functions, using just first order information. When I say first order information, it is the first derivative or in higher dimension function, it is the gradient which is just a vector of partial derivatives. So, what is the gradient descent algorithm.

So, you start at some point some guess for your solution, which let us call that guess is w_{naught} , which is some arbitrary points in d dimension and then you compute the gradient of this the objective function at the point w_{naught} where you are. So, the gradient would tell you the direction in which you have to move such that the function increases the most. So, the direction where the function gives the highest increase is what its given by your gradient.

Now, we want to minimize the function. So, naturally gradient descent would tell you well move in the direction opposite to what the gradient is pointing, so, the gradient would tell you which direction you have to climb the mountain, the negative of the gradient to tell you direction in which you are to move such that before you reach the bottom of the mountain, so, you kind of reduce your function value.

(Refer Slide Time: 06:59)

The slide contains handwritten notes in blue and red ink. At the top right, it says "is expensive $O(d^3)$ ". Below that, a bullet point states: "We know w^* is the solution of unconstrained optimization". Another bullet point says: "We can apply GRADIENT DESCENT." Below this, the update rule is written: $w^{t+1} = w^t - \eta \nabla F(w^t)$. Arrows point from the terms in the equation to their meanings: η is labeled "Scalar step size" and $\nabla F(w^t)$ is labeled "Gradient at w^t ". In the bottom left corner, there is a small video inset showing a man speaking. In the bottom right corner, there is a logo for "IIT Madras B.S. Degree".

So, the update rule would be w_{t+1} would be w_t minus the gradient come off the function any general function evaluated at w_t but then you do not take a full step necessarily in the negative gradient direction the negative the minus says that you have to move in the negative gradient direction this is the gradient, gradient at w_t but then you move some scalar times the negative gradient where scalar tells you what, how much step should I take in the direction of pointed by the negative gradient. So, this is the step size basically.

Now, the property of this algorithm is that if you choose to step size carefully, then if you keep repeating this process, this is an iterative algorithm you move take a small step in a gradient direction at that new point, the gradient of the function might point you in a different direction you keep doing this, eventually you will reach what is called as a local minima of the function. Now, a local minima is a point where the gradient becomes 0. And that does not mean that it is necessarily the global minima.

(Refer Slide Time: 08:16)

is expensive $O(d^3)$

- We know w^* is the solution of an unconstrained optimization
- We can apply GRADIENT DESCENT.

$$w^{t+1} = w^t - \eta^t \nabla F(w^t)$$

Scalar step size η^t $\nabla F(w^t)$ gradient at w^t

IIT Madras
B.S. Degree

For example, you might have a function like this, where this is a local minima, this is a local minima, whereas, this is the global minimum of this function, which means, if we start at this point, then if you start at this point, then you will move in this direction and you will reach this point which is a local minima and then your algorithm does not proceed any further, it kind of stops at that point, because the gradient becomes 0 and then this update rule does not proceed any further.

However, if your function is nice, not, vaguely like this, but then if the function is nice, something like this, then yes, your gradient descent algorithm, in fact, does not matter where you start will converge to the global minima, because there is only one global minima, or every local minima is also a global minimum.

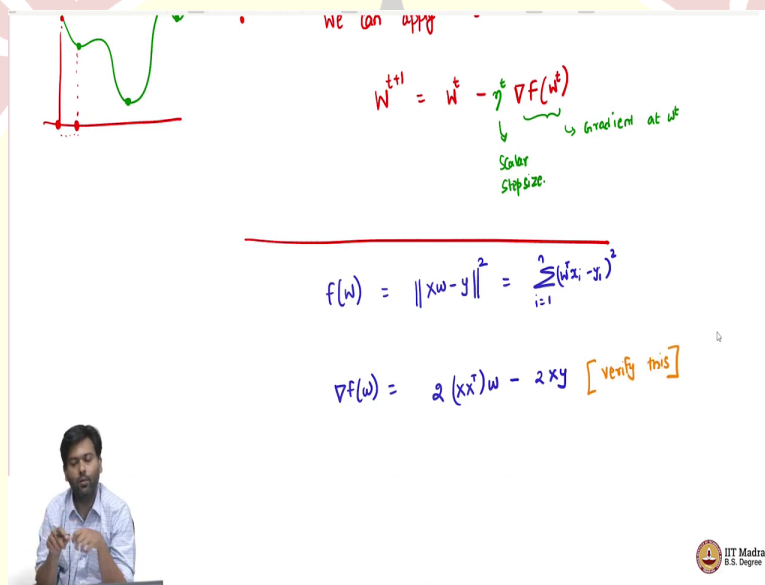
And it turns out that, for the function that we care about, which is the sum of the squared errors, is, in fact, the function that looks like the which does not have wiggles it is a quadratic function and quadratic functions are a subset of a class of functions called convex functions, which have a unique global minima in this case.

So, which means you are, in which is for which all local minima are global minima, which means that your gradient descent algorithm will in fact converge to the optimal point. So, so we

can apply this, yes, this is an alternate way to solve the same problem. The question is does it solve our original problem, which is like we had a problem of computing inverses.

Now, we are saying well, I do not care about one shot solution, so I do not need an equation, but then I will do it in an iterative fashion. But then my inverse computation should go away. Now, question is, is it going away in this particular example? Let us see that, you may want to pause and think about this and I will tell you what the solution is at this point.

(Refer Slide Time: 10:14)



we can apply

$$w^{t+1} = w^t - \eta \nabla F(w^t)$$

Scalar Step Size η Gradient at w^t

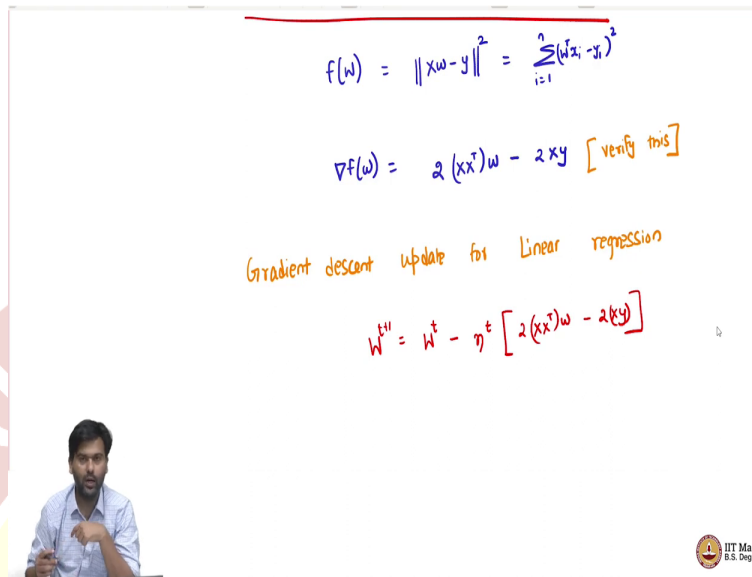
$$f(w) = \|xw - y\|^2 = \sum_{i=1}^n (w^T x_i - y_i)^2$$

$$\nabla f(w) = 2 (xx^T)w - 2xy \quad [\text{verify this}]$$

IIT Madras
B.S. Degree

So, what is the function that we care about. So, our function is, $f(w) = \|xw - y\|^2$, which is simply $\sum_{i=1}^n (w^T x_i - y_i)^2$, we saw this already that these two things are the same. Now, for this function, one can compute the gradient. And the gradient would look something like $2(xx^T)w - 2xy$. Now, this is a simple exercise, please verify this for yourself, you just have to take the derivative of this function with respect to each component of w and then get a vector of partial derivatives and that would give you exactly this. So, it is a good exercise to verify this, if you are not that comfortable with partial derivatives, and so on, it is a good exercise to try this.

(Refer Slide Time: 11:18)


$$f(w) = \|xw - y\|^2 = \sum_{i=1}^n (w x_i - y_i)^2$$
$$\nabla f(w) = 2 (x x^T) w - 2 x y \quad [\text{verify this}]$$

Gradient descent update for linear regression

$$w^{t+1} = w^t - \eta^t [2 (x x^T) w - 2 x y]$$

So, which means what does this tell us, this tells us that gradient descent update rule for our linear regression for linear regression is going to look like this. w_{t+1} is going to be $w_t - \eta^t [2(x x^T) w - 2(xy)]$. This 2 is not so important, you might have added a half in the original function, and I could have thought of this $f(w)$ itself as half, and then the 2 would cancel out, that is that is usual practice but then here, I am not doing that, but it would not really matter if the 2 can be absorbed in the constant if you will, the steps are the same.

Okay so, the good thing here is that there is no inverse computation happening here. As you can see, w_{t+1} is w_t the current w minus some scalar times some quantity, which does not require the inverse to be computed. So, we are not computing the inverse at all it we will converge to w^* , which would have essentially computed the inverse and then done an $(x x^T)^{-1} x y$.

So, we are mimicking that not in one step in multiple steps via the gradient rule, but eventually you will converge to the same w^* , which is a good thing. So, computationally, we are never computing the inverse. So, even if d is much, much larger, even if d is in millions, number of features that you have is not millions, can still run this algorithm that is what this this kind of tells us if you were inverse computation is a bottleneck.

Now, what might also happen on the other hand is one might ask the question, well, if d is large we have a problem, you have kind of solved that by going to this gradient descent algorithm, what if n is large.

(Refer Slide Time: 13:23)

The slide contains the following handwritten text:

$$W^{t+1} = W^t - \eta^t \left[2 \underbrace{(XX^T)w}_{\text{hundreds of millions}}, - 2ky \right]$$

- What if n is large [hundreds of millions], might want to avoid XX^T
- How to adapt gradient descent?

In the bottom left corner, a man is visible, gesturing with his hand. In the bottom right corner, there is a logo for IIT Madras B.S. Degree.

So, the next question is what if n is large? Well, n is the number of data points, so, let us say hundreds of millions. So, now, what is there is there an issue that that we want to solve here, well, now remember gradient descent is an iterative process. So, at every round you need to compute the gradient. So, which means that you need to compute, the gradient requires xx^T to be computed, and xx^T is a $d \times d$ matrix, but then it came from $d \times n$ times $n \times d$ matrix.

So, there is an n dependency hiding inside this xx^T itself. And if n itself is in billions, then even computing this xx^T might actually be hard thing to do. So, we may want to might want to avoid xx^T computation itself, not just the inverse of it, but even xx^T might be hard to compute, if you have, huge number of data points.

So, now the question is, well, then it appears that we cannot even run gradient descent because it needs xx^T and that is a that might be computationally expensive also, the question then we ask is, how to adapt gradient descent to this situation. Can we somehow mimic something like gradient descent to get rid of this problem of computing xx^T ?

Well, the surprising answer to this is yes, you can get rid of this both the d problem and the n problem, the d problem via using a gradient descent algorithm, the n problem via doing some, variant of the gradient descent algorithm, which is a very popular algorithm that people use in practice all the time.

(Refer Slide Time: 15:39)

STOCHASTIC GRADIENT DESCENT

for $t=1, \dots, T$

- At each step, sample a bunch of datapoints (R) uniformly at random from the set of all points

IIT Madras
U.S. Degree

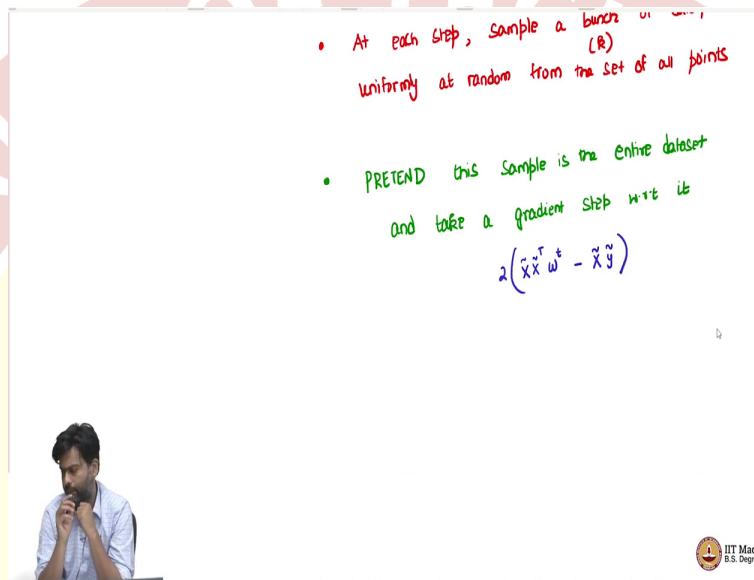
And let us look at what this variant is. This variant is what is called as stochastic gradient descent. It is a variant of gradient descent, there are several variants, I am going to give you one simple variant of gradient descent here. Like gradient this entities and iterative algorithm for $t = 1, \dots, T$, you can have a fixed number of rounds that you want to run this algorithm for, or you might have some convergence criteria, that is similar to what gradient descent would do.

Nevertheless, we let us say, we have a budget number of rounds for which we want to run this algorithm for. Now, what you do in stochastic gradient descent differently from gradient descent is that the first thing you do is at each step, I am giving you a pseudo code at each step, sample, a bunch of let us say some k of data points, uniformly at random from the set of all data points.

What does this mean? This means that you might have a billion data points to begin with. Now, what we are going to do is, let us say our k is some 100. Now, we are going to uniformly sample 100 data points from this billion data points.

Now, that is that can be efficiently done, because you are just trying to get uniformly 100 indices, from the indices from one to billion, which can be done very quickly. And then once you have these 100 indices, you can go and fetch these 100 data points if you have if you have stored your data points in a in a reasonably good way, so that is not a big issue.

(Refer Slide Time: 17:35)



- At each step, sample a bunch of size (B) uniformly at random from the set of all points
- PRETEND this sample is the entire dataset and take a gradient step wrt it

$$2(\tilde{x}\tilde{x}^T w^t - \tilde{x}\tilde{y})$$

IIT Madras
B.S. Degree

Now, what you do, the next step is something interesting, you are now, you are now do not have billion data points at your hand, you only have 100 data points. Now, you pretend as if this sample is the entire data set, so is the entire data set and you take a gradient step with respect to this step, this sample, so with respect to it. So, which means our gradient at the t step is going to look like the following 2 times not xx^T , it is going to be $(\tilde{x}\tilde{x}^T w^t - \tilde{x}\tilde{y})$. Now, what is $\tilde{x}\tilde{x}^T$ is just the just the bunch of data points that we sampled in this round. So, it is not a $n \times d$ matrix anymore. It is just $100 \times d$ matrix, not a billion $\times d$, it is just $100 \times d$ because you are only sampling 100 data points.

Now, what you are doing essentially is that you are pretending that in this step, you only have 100 data points to look at, and we will take a gradient step with respect to only these 100 data points as if this was our dataset. And as if we are trying to minimize the squared error only with respect to this data set at this point. So, you take a gradient step that works only well for this data set.

(Refer Slide Time: 19:13)

• At each step, sample a bunch of points (R) uniformly at random from the set of all points

• PRETEND this sample is the entire dataset and take a gradient step wrt it

$$\frac{1}{2} (\tilde{x} \tilde{x}^T w - \tilde{x} \tilde{y})$$

↳ manageable because $\tilde{x}$ is small

end

IIT Madras
B.S. Degree

So, of course, this is manageable, this is manageable. Because you know, because you have $\tilde{x}$ which is $d \times k$, so where k is small, k is the number of sample points, number of sample points. That is it that is the algorithm so, end. So, so, this is the stochastic gradient descent algorithm, you start, you run this for t rounds.

Note that at every round, you are sampling k points, which means these k points are not the same at every round, that is the most important thing, every round, you get a different sample though uniformly distributed from the set of all points, and then you are moving in the gradient direction along for with respect to the sample that you have got at this round.

Now, you can argue that this sample and the gradient that comes out of the sample is a noisy gradient. So, you had to come to the original gradient. But because you are doing this uniform sampling, you somehow have a noisy version of the original gradient, and you are moving in a noisy direction.

So, the question is, well, we are moving in noisy directions, alright but then over time, somehow, does this noise kind of cancel out and then are we overall making progress towards the original w^* that gradient descent would make progress towards the surprising and the good answer good news is that the answer to this question is yes.

So, the with a single small caviar that here, in gradient descent, typically you do this, you start with some w not, you move in the gradient direction to $w_1, w_2 \dots w_T$, and after the end of T rounds, that w_T is what you would output. Now, because here at every round, there is a chance that you might get a noisy gradient, and you might get an unlucky direction to move, just taking the last w_T may not necessarily be a good idea, because you might have gotten a very unlucky sample in the last round, and you might move in some arbitrary direction, which is not really which is taking you away from the minimizer.

(Refer Slide Time: 21:40)

End

$2(\bar{x}\bar{x}^T w - xy)$

↳ manageable because $\bar{x} \in \mathbb{R}^n$ & $w \in \mathbb{R}^n$ is a scalar

After T rounds, use

$$\bar{w}_{SGD} = \frac{1}{T} \sum_{t=1}^T w^t$$

as opposed to w^T as in Standard Gradient descent

Guaranteed to converge to optima with high probability

IIT Madras
B.S. Degree

So, instead, what you do is after T rounds, you typically use $\bar{w}^T$, given by a stochastic gradient descent algorithm as just the average of all the w 's that you have seen so far, w . So, this is one way to, as opposed to, the standard w^T that you would have used in as in standard gradient descent, so instead of using w^T now, you are going to use the average of all the w 's that you have seen so far.

Now, what is the what can we guarantee about this stochastic gradient descent's output w_T is that you can guarantee this is guaranteed to converge or to Optima, which is w^* with high probability.

Of course, because there is a notion of randomness involved in picking the data points, there is a noisy gradient involved, we have to deal with probabilities here, but what is the guarantee that

one can show is that you can converge to what you want the w^* with very high chance, this is a good thing.

Now, what you have done is you have kind of gotten away both the problems of d and n by first switching to gradient descent and then by switching to the stochastic version of it and this works so well in practice. So, this k is what is called as a batch size and now, depending on how much computation you can deal with you can increase k or decrease k .

So, the guarantee would it works independent of what the batch size is, of course, the batch size will determine how many more rounds that you may want than standard gradient descent, because the smaller the batch size, slightly more number of rounds that you may have to run your algorithm for to get to the minimizer.

Nevertheless, this is an extra computation, which is much more manageable than computing xx^T which might be computationally very expensive especially if n is too large. So, this algorithm is called as stochastic gradient descent and this is what I want to discuss in brief about the computational considerations of an algorithm such as linear regression.

So, we will, we will ask a couple of more questions about linear regression. And the next topic that we would like to talk about is why linear regression. So, why did we choose linear regression, and why can't we have a more general or maybe a non (nes) not necessarily a linear relation between the features that explains the output variable y , what if for instance, if it is a quadratic relation or something like that, so that will take us into what is called as kernel regression like how we did a kernel PCA now we are going to look at a kernelized version of linear regression which is very similar so, we will try to understand the kernel regression next.